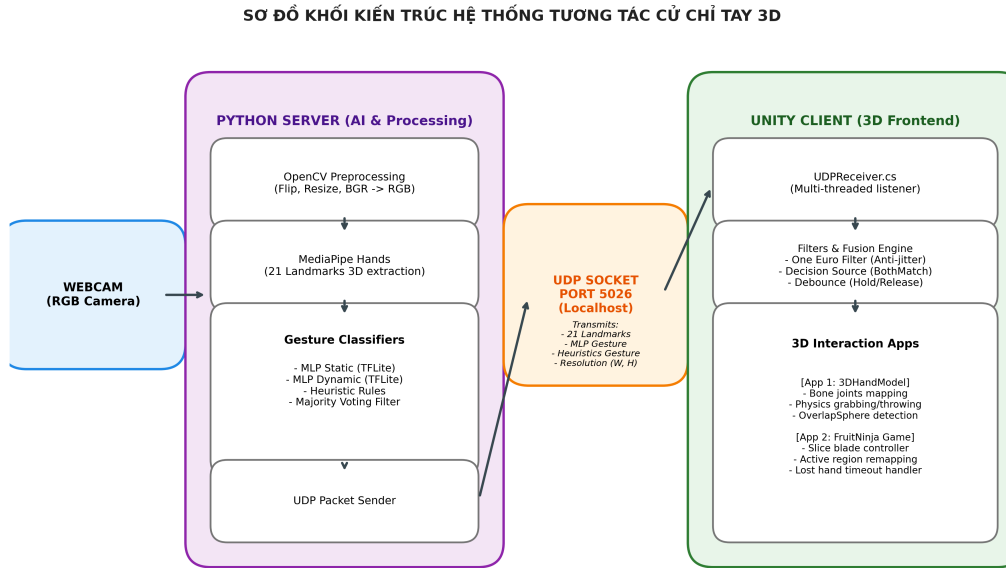


Tổng quan chương

Chương 2 đã thiết lập các cơ sở lý thuyết toán học và giải thuật vững chắc cho hệ thống. Chương 3 này sẽ tập trung mô tả chi tiết quá trình hiện thực hóa kiến trúc phần mềm, lập trình các kịch bản tương tác trong Unity và đánh giá các kết quả thực nghiệm thu được. Nội dung chương bao gồm bốn phần chính: (i) Thiết kế kiến trúc hệ thống và giao thức truyền dữ liệu; (ii) Cài đặt module xử lý Python backend; (iii) Lập trình các ứng dụng tương tác 3D frontend trong Unity; và (iv) Đánh giá hiệu năng hệ thống về độ chính xác phân loại cử chỉ, tốc độ xử lý FPS, độ trễ hệ thống và hiệu quả lọc nhiễu của bộ lọc One Euro Filter.

0.1 Kiến trúc hệ thống và giao thức truyền dữ liệu

Hệ thống tương tác cử chỉ tay thời gian thực được thiết kế theo kiến trúc Client-Server phân tán nhằm tối ưu hóa tải tính toán. Việc tách biệt luồng xử lý AI (phía Python Server) và luồng kết xuất đồ họa mô phỏng 3D (phía Unity Client) giúp hệ thống duy trì hiệu năng cao trên các cấu hình máy tính cá nhân thông thường. Sơ đồ khối kiến trúc hệ thống được thể hiện trong Hình 0.1.



Hình 0.1: Sơ đồ khối kiến trúc hệ thống tương tác cử chỉ tay thời gian thực

Giao thức truyền dữ liệu giữa Python và Unity được xây dựng trên nền tảng socket mạng UDP. Phía Python mã hóa dữ liệu thành một chuỗi ký tự UTF-8 phân tách bằng dấu phẩy và gửi đến địa chỉ mạng cục bộ `127.0.0.1` trên cổng `5026`. Cấu trúc gói tin dữ liệu được đóng gói theo phương trình (1) như sau:

$$\text{Data} = [x_0, y_0, z_0, \dots, x_{20}, y_{20}, z_{20}, \text{GestureName}, W, H, \text{RuleGestureName}] \quad (1)$$

Trong đó: (i) x_i, y_i, z_i ($i \in [0, 20]$) là tọa độ không gian 3D thô của 21 điểm landmark khớp bàn tay do MediaPipe trích xuất (với trục y được biến đổi ngược chiều để phù hợp với hệ tọa độ Unity); (ii) GestureName là nhãn cử chỉ tĩnh được nhận dạng bởi mô hình học máy (ví dụ: "Open", "Close", "Pointer"); (iii) W cùng H tương ứng là chiều rộng và chiều cao thực tế của khung hình camera đầu vào, được gửi kèm để phía Unity thực hiện chuẩn hóa tỉ lệ độc lập với độ phân giải; và (iv) RuleGestureName là nhãn dự đoán cử chỉ tĩnh dựa trên thuật toán luật hình học Heuristics, phục vụ cho cơ chế dung hợp và xác thực quyết định phía Unity.

Trong phiên bản hiện tại, gói UDP sử dụng cho Unity truyền tọa độ landmark, nhãn cử chỉ tĩnh (từ cả MLP và Heuristics) cùng kích thước khung hình W, H . Kết quả nhận dạng cử chỉ động được xử lý và đánh giá hoàn toàn ở phía Python, chưa được tích hợp trực tiếp vào gói dữ liệu điều khiển Unity.

Để truyền dữ liệu, phía Python chuyển mảng dữ liệu thành một chuỗi văn bản dạng danh sách đơn giản, ví dụ `"[x_0, y_0, z_0, ..., 'Gesture-Name', W, H, 'RuleGestureName']"`, sau đó gửi đi dưới dạng mảng byte mã hóa UTF-8. Phía Unity Client sử dụng lớp `UDPReceive.cs` để nhận luồng byte này qua một luồng chạy nền độc lập. Khi dữ liệu được nhận, chuỗi ký tự được xử lý lại ở luồng chính `Update()` bằng cách loại bỏ các ký tự ngoặc vuông ở hai đầu, tách các phần tử bằng dấu phẩy và chuyển từng phần tử về kiểu dữ liệu tương ứng.

0.2 Hiện thực hóa module xử lý Python Backend

Phần mềm phía Python được triển khai thông qua file mã nguồn `main.py` tích hợp đầy đủ ba chế độ hoạt động:

1. **Chế độ Dự đoán (Predict Mode - Mode 0):** Chế độ hoạt động chính của hệ thống. Luồng webcam từ camera được OpenCV đọc liên tục. Nhằm tối ưu hóa tốc độ xử lý của MediaPipe, độ phân giải camera được giảm xuống mức 960×540 pixel. Tọa độ của 21 điểm landmark sau khi trích xuất được chuẩn hóa theo công thức (??) và tiếp tục được đưa vào mô hình `GestureClassifier` để nhận diện cử chỉ tĩnh. Đồng thời, lịch sử tọa độ ngón trỏ được tích lũy vào hàng đợi để đưa vào mô hình `DynamicClassifier` nhận diện cử chỉ động.
2. **Chế độ Thu thập cử chỉ tĩnh (Collect Static Mode - Mode 1):** Cho phép người dùng nhấn các phím tương ứng với mã nhãn cử chỉ để thu thập dữ liệu landmark chuẩn hóa tương ứng với các lớp cử chỉ (sử dụng 6 lớp tĩnh: Open, Close, Pointer, OK, Peace, ThumbsUp) và ghi trực tiếp vào tệp `gesture_classification.csv`.

3. **Chế độ Thu thập cử chỉ động (Collect Dynamic Mode - Mode 2):** Thu thập chuỗi hành trình di chuyển của ngón trỏ và lưu vào tệp dữ liệu `dynamic_gesture.csv` nhằm phục vụ cho quá trình huấn luyện mô hình cử chỉ động.

0.3 Lập trình tích hợp tương tác 3D trong Unity

Phía Unity nhận dữ liệu truyền từ Python thông qua lớp `UDPReceive.cs` chạy trên một luồng nền riêng biệt (background thread) để tránh gây nghẽn luồng kết xuất đồ họa chính. Hệ thống được triển khai trên hai ứng dụng độc lập:

0.3.1 Ứng dụng tương tác vật lý `3DHandModel`

Ứng dụng này ánh xạ tọa độ 21 điểm nhận được từ Server để tái tạo bộ khung xương bàn tay ảo và điều khiển các hoạt động tương tác. Phép chuyển đổi hệ tọa độ được kịch bản `HandTracking.cs` thực hiện thông qua hai chế độ hoạt động chính:

- **Chế độ ánh xạ tỉ lệ 1:1 mặc định:** Tọa độ thô $x_{raw}, y_{raw}, z_{raw}$ nhận được từ Python được chuyển trực tiếp sang không gian thế giới (world unit) và co giãn qua hệ số chia `coordinateDivisor` (mặc định bằng 75). Hệ tọa độ được lật dọc (mirror) trên trục hoành để hiển thị tự nhiên qua biểu thức $x_{unity} = 7 - x_{raw}/divisor$, các trục còn lại tương ứng là $y_{unity} = y_{raw}/divisor$ và $z_{unity} = z_{raw}/divisor$.
- **Chế độ khuếch đại tầm với thích nghi (Reach Remap):** Khi kích hoạt `enableReachRemap`, hệ thống tách biệt hành vi di chuyển (tầm với) khỏi kích thước của bàn tay ảo. Tọa độ của điểm cổ tay (Wrist - điểm số 0) được chuẩn hóa về khoảng $[0, 1]$ dựa theo kích thước khung hình camera thật (W, H), sau đó ánh xạ phi tuyến vào vùng thế giới ảo rộng lớn xác định bởi tâm `reachCenter` và phạm vi quét `reachRange`. Tọa độ 20 điểm khớp còn lại được tính bằng khoảng lệch (offset) hình học tương đối so với cổ tay rồi chia cho hệ số `coordinateDivisor`. Cơ chế này giúp kích thước bàn tay ảo luôn giữ tỷ lệ tự nhiên cố định, đồng thời người dùng chỉ cần di chuyển tay nhẹ nhàng ngoài đời thực vẫn có thể tiếp cận và thao tác toàn bộ các vùng xa nhất của hộp chứa vật lý trong Unity.

Việc liên kết giữa các khớp xương được thực hiện tự động thông qua các đoạn thẳng trong thành phần `Line.cs` để tạo nên bộ khung xương trực quan. Đồng thời, module `GrabbableSpawner.cs` hỗ trợ cơ chế sinh tự động các khối hộp vật lý ngẫu nhiên hoặc theo vị trí xác định trước. Kịch bản hỗ trợ phương thức `SpawnBatch()` chạy trực tiếp tại runtime (Play Mode) thông qua nút Inspector Context Menu, cho phép liên tục sinh thêm các batch vật thể mới mà không cần

reset trạng thái ứng dụng.

Để thực hiện tương tác cầm nắm, kịch bản `HandGrabber.cs` thực hiện ba chức năng chính: thứ nhất là xác định vị trí lòng bàn tay thông qua việc tính toán trọng tâm (centroid) và hướng xoay (rotation) của lòng bàn tay dựa trên 5 điểm mốc chính gồm cổ tay 0, gốc ngón trỏ 5, gốc ngón giữa 9, gốc ngón áp út 13 và gốc ngón út 17 (trong đó hướng lòng bàn tay được xác định bằng tích có hướng giữa hai vectơ ngón trỏ và ngón út); thứ hai là phát hiện va chạm bằng cách sử dụng các quả cầu kiểm tra va chạm vật lý (Overlap Sphere) với bán kính `pointRadius = 0.35` quanh các điểm mốc ngón tay để xác định các vật thể có thuộc tính `Grabbable` đang nằm trong tầm với; và thứ ba là thực hiện các hành vi cầm nắm và ném vật lý khi nhận được cử chỉ tĩnh "Close" từ phía Python Server để gán vật thể làm con của lòng bàn tay (Grab), giải phóng vật thể khi cử chỉ chuyển trạng thái (Release), đồng thời áp dụng lực đẩy vật lý (`Rigidbody.velocity`) dựa trên vận tốc ném trung bình của 10 khung hình gần nhất để tạo cảm giác ném tự nhiên.

Để tối ưu hóa trải nghiệm tương tác vật lý và triệt tiêu các lỗi nhận dạng sai từ mô hình AI, ứng dụng tích hợp ba cơ chế nâng cao trực tiếp trong kịch bản điều khiển:

- **Dung hợp cử chỉ thích nghi (Gesture Source):** Cho phép lựa chọn giữa việc sử dụng độc lập mô hình MLP, thuật toán Heuristics, cơ chế dự phòng thích nghi (*EitherMatch*) hoặc cơ chế đồng thuận tuyệt đối (*BothMatch*). Chế độ *BothMatch* chỉ kích hoạt hành động cầm nắm khi cả mô hình MLP và thuật toán Heuristics đều đồng thời trả về nhãn "Close", giúp loại bỏ hoàn toàn các trường hợp vô tình kích hoạt cầm/ném do sai số của mô hình học máy.
- **Bộ lọc danh sách đen (Ignored Gestures):** Thiết lập danh sách các cử chỉ nhạy cảm dễ gây nhiễu để hệ thống bỏ qua và giữ nguyên trạng thái điều khiển trước đó khi nhận diện nhầm các nhãn này.
- **Cơ chế chống trễ và duy trì trạng thái (Grab/Release Debounce):** Cài đặt ngưỡng thời gian duy trì liên tục trạng thái cử chỉ trước khi chuyển đổi logic thực tế (0.15 giây đối với hành vi bắt đầu cầm nắm và 0.5 giây đối với hành vi thả rơi vật thể). Nhờ đó, nếu mô hình AI bị nhảy nhãn tức thời trong vài khung hình do tay người dùng chuyển động nhanh, vật thể vẫn được giữ chắc chắn mà không bị thả rơi tự do ngoài ý muốn.

HÌNH ẢNH MINH HỌA
(Hãy chụp màn hình ứng dụng 3DHandModel và ghi đè vào tệp:
BTL Template/Hinhve/unity_hand_model.png)

Hình 0.2: Giao diện mô phỏng xương bàn tay 3D và hành động cầm nắm vật thể ảo trong Unity

0.3.2 Ứng dụng trò chơi FruitNinja điều khiển bằng cử chỉ

Trò chơi FruitNinja được điều khiển bằng vị trí đầu ngón trỏ và trạng thái cử chỉ Pointer thay cho thao tác chuột truyền thống. Quá trình xử lý bao gồm hai nhiệm vụ chính: (i) ánh xạ tọa độ màn hình (`HandInput.cs`) bằng cách lấy duy nhất tọa độ của điểm đầu ngón trỏ (landmark 8), chuẩn hóa về khoảng $[0, 1]$ dựa trên kích thước camera, áp dụng bộ lọc thích nghi One Euro Filter với các tham số được sử dụng trong thử nghiệm là $\text{minCutoff} = 1.0$ và $\text{beta} = 0.4$ để giảm rung lắc (jitter) rồi remap ra kích thước màn hình game (`Screen.width`, `Screen.height`); và (ii) kích hoạt lưỡi dao (`Blade.cs`) khi nhận cử chỉ "Pointer" từ phía Python Server, lưỡi dao được kích hoạt và di chuyển theo vị trí ngón trỏ, đồng thời tạo vệt chém (Trail Renderer) và bật collider để cắt các quả bay lên khi vận tốc di chuyển vượt qua ngưỡng $\text{minSliceVelocity} = 0.01$.

Bên cạnh đó, nhằm tăng tính tiện dụng cho trò chơi thực tế, module `HandInput.cs` tích hợp thêm hai giải thuật tối ưu hóa động học:

- **Ánh xạ vùng hoạt động giới hạn (Active Region Remapping):** Thực hiện ánh xạ phi tuyến một phân vùng tọa độ thô thu hẹp từ camera (ví dụ khoảng $[0.3, 0.7]$) ra toàn bộ độ phân giải màn hình game thông qua hàm nội suy Remap. Cơ chế này giúp người dùng chỉ cần di chuyển tay trong một phạm vi nhỏ trước camera vẫn có thể quét hết các góc màn hình, giảm thiểu đáng kể sự mệt mỏi vật lý khi tương tác lâu.
- **Tự động xử lý mất dấu tay (Lost Hand Timeout):** Thiết lập bộ giám sát thời gian trễ 0.2 giây. Khi không nhận được gói tin UDP hợp lệ từ Server vượt quá ngưỡng này, hệ thống tự động giải phóng lưỡi dao và đưa trạng thái cử chỉ về

mặc định, tránh việc lưỡi dao bị đứng đơ tại vị trí biên cuối cùng khi người dùng đưa tay ra khỏi khung hình camera.



Hình 0.3: Giao diện trò chơi FruitNinja điều khiển bằng cử chỉ ngón trỏ (Pointer)

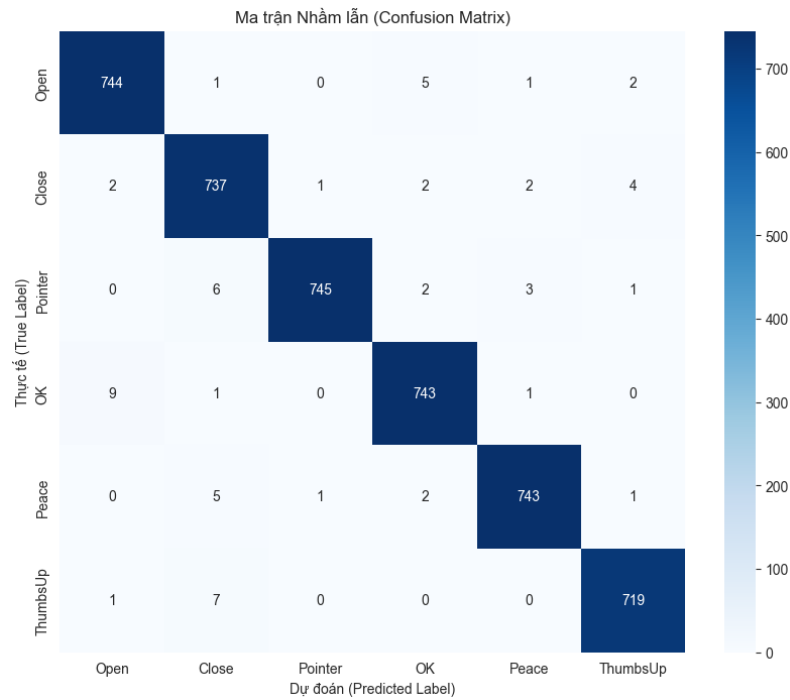
0.4 Đánh giá kết quả thực nghiệm

Quá trình kiểm thử hệ thống được thực hiện trên máy tính cấu hình văn phòng phổ thông sử dụng CPU Intel Core i5 và webcam tích hợp 720p.

0.4.1 Kết quả huấn luyện và Độ chính xác mô hình học máy

Hiệu năng nhận dạng của hệ thống phụ thuộc hoàn toàn vào kết quả huấn luyện của hai mô hình mạng nơ-ron nhân tạo MLP trên tập dữ liệu cử chỉ tĩnh và cử chỉ động thực nghiệm.

Đối với mô hình phân loại cử chỉ tĩnh sử dụng mạng nơ-ron MLP, tập dữ liệu thực tế thu thập gồm 13.471 mẫu huấn luyện và 4.491 mẫu kiểm thử phân tầng (stratified test set). Cấu trúc mạng MLP gồm lớp đầu vào 42 đặc trưng tọa độ landmark chuẩn hóa, hai lớp ẩn có kích thước lần lượt là 64 nơ-ron (kèm dropout 0.3) và 32 nơ-ron, lớp đầu ra sử dụng hàm Softmax dự đoán 6 lớp cử chỉ (Open, Close, Pointer, OK, Peace, ThumbsUp). Quá trình huấn luyện sử dụng bộ tối ưu hóa Adam với tốc độ học 0.001, chạy tối đa 500 epochs và dừng sớm ở epoch 104 do hàm val_loss không tiếp tục cải thiện. Bộ trọng số tối ưu nhất được khôi phục tại epoch 84 với val_loss đạt 0.0431 và độ chính xác val_accuracy đạt 98.78%. Khi đánh giá trên tập kiểm thử độc lập, mô hình đạt độ chính xác toàn cục (Accuracy) khoảng 99% trên tập kiểm thử. Kết quả dự đoán chi tiết đối với từng lớp cử chỉ tĩnh của mô hình MLP được thể hiện qua ma trận nhầm lẫn ở Hình 0.4 và báo cáo kết quả chi tiết ở Bảng 1.

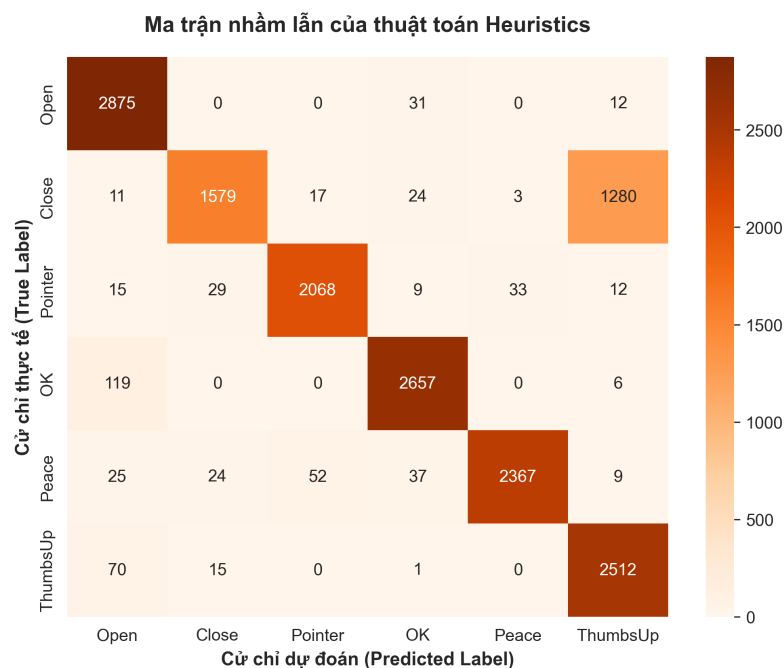


Hình 0.4: Ma trận nhầm lẫn của mô hình phân loại cử chỉ tĩnh bằng mạng MLP

Lớp cử chỉ tĩnh (MLP)	Precision	Recall	F1-Score	Support
Open	0.98	0.99	0.99	753
Close	0.97	0.99	0.98	748
Pointer	1.00	0.98	0.99	757
OK	0.99	0.99	0.99	754
Peace	0.99	0.99	0.99	752
ThumbsUp	0.99	0.99	0.99	727
Đoán đúng toàn cục (Accuracy)	0.99			4.491

Bảng 1: Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ tĩnh MLP

Để làm nổi bật ưu thế của việc sử dụng mạng nơ-ron, hệ thống tiến hành đánh giá đối chứng thuật toán hình học Heuristics (định nghĩa trong tệp mã nguồn `rule_gesture.py`) trên toàn bộ tập dữ liệu cử chỉ tĩnh gồm 17.962 mẫu. Kết quả thực nghiệm cho thấy thuật toán Heuristics chỉ đạt độ chính xác toàn cục (Accuracy) là 78.27%. Trong đó, lớp cử chỉ Close có độ phủ (Recall) thấp nhất là 53% với 1.280 lần bị phân loại nhầm thành ThumbsUp do ngón cái gấp lại vẫn bị công thức khoảng cách cứng nhắc nhận diện nhầm là duỗi. Lớp cử chỉ Pointer đạt F1-score là 80% do nhiều góc xoay tay bị thuật toán nhận diện nhầm thành trạng thái không xác định (None). Kết quả dự đoán chi tiết của thuật toán Heuristics được thể hiện qua ma trận nhầm lẫn ở Hình 0.5 và chi tiết chỉ số từng lớp ở Bảng 2.



Hình 0.5: Ma trận nhầm lẫn của thuật toán phân loại cử chỉ tĩnh bằng luật Heuristics

Lớp cử chỉ tĩnh (Heuristics)	Precision	Recall	F1-Score	Support
Open	0.92	0.95	0.94	3.011
Close	0.96	0.53	0.68	2.991
Pointer	0.97	0.68	0.80	3.027
OK	0.96	0.88	0.92	3.015
Peace	0.99	0.79	0.87	3.008
ThumbsUp	0.66	0.86	0.75	2.910
Đoán đúng toàn cục (Accuracy)	0.78			17.962

Bảng 2: Báo cáo kết quả phân loại chi tiết (Classification Report) của thuật toán hình học Heuristics

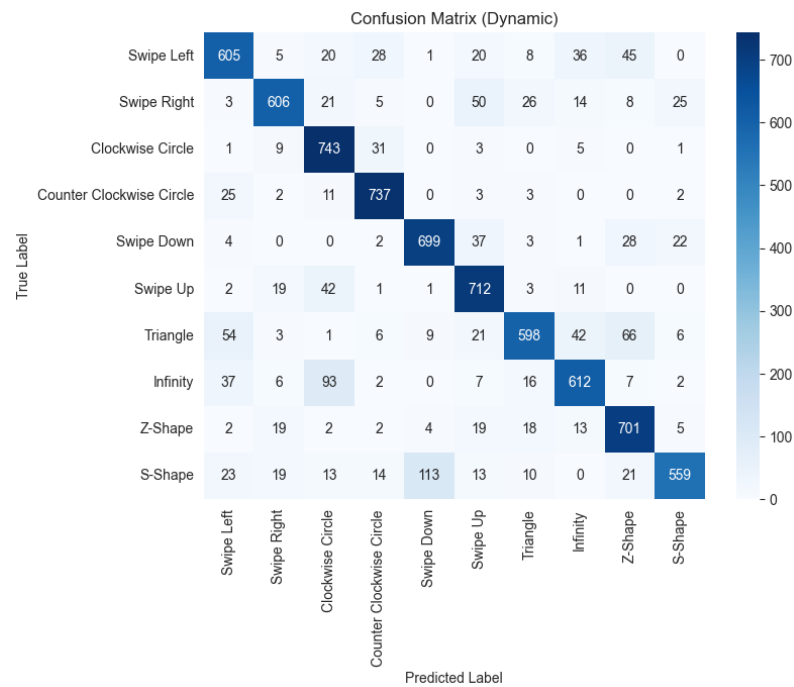
Để có cái nhìn tổng quan về cả hai giải thuật phân loại cử chỉ tĩnh, Bảng 3 tổng hợp các đặc tính hiệu năng chính của hai phương pháp:

Chỉ số so sánh (Cử chỉ tĩnh)	Mô hình học máy MLP	Thuật toán hình học Heuristics
Độ chính xác toàn cục (Accuracy)	99.0%	78.27%
F1-score trung bình (Macro Avg)	0.99	0.83
Lớp nhận diện tốt nhất (F1-score)	Open/Pointer/OK/Peace/ThumbsUp (0.99)	Open (0.94) / OK (0.92)
Lớp nhận diện kém nhất (F1-score)	Close (0.98)	Close (0.68) / ThumbsUp (0.75)
Thời gian huấn luyện mô hình	10 giây (84 epochs)	Không cần huấn luyện
Khả năng mở rộng cử chỉ mới	Rất cao (chỉ cần thêm nhãn và mẫu)	Thấp (phải thiết kế lại toàn bộ quy tắc)

Bảng 3: Bảng so sánh hiệu năng giữa mô hình học máy MLP và thuật toán hình học Heuristics

Đối với mô hình phân loại cử chỉ động dựa trên quỹ đạo, tập dữ liệu thu thập gồm 23.538 chuỗi quỹ đạo huấn luyện và 7.847 chuỗi kiểm thử thuộc 10 lớp cử

chỉ động chuyển động. Mạng MLP cho cử chỉ động nhận đầu vào là vectơ 64 chiều (chuỗi tọa độ 32 khung hình gần nhất của ngón trỏ), đi qua lớp Dropout tỉ lệ 0.2, lớp ẩn 128 nơ-ron (kèm dropout 0.4), lớp ẩn 64 nơ-ron và lớp đầu ra Softmax 10 lớp. Mô hình được thiết lập huấn luyện tối đa 1.000 epochs, thực hiện dừng sớm ở epoch 109 và khôi phục bộ trọng số tốt nhất ở epoch 89 với val_loss đạt 0.4421 và val_accuracy đạt 86.12%. Đánh giá trên tập kiểm thử độc lập 7.847 mẫu cho thấy mô hình đạt độ chính xác trung bình khoảng 84.0% trên tập kiểm thử độc lập. Ma trận nhầm lẫn ở Hình 0.6 thể hiện chi tiết kết quả phân loại, giúp quan sát các lớp dễ nhầm lẫn. Các chỉ số đánh giá chi tiết (Precision, Recall, F1-Score và Support) của từng lớp cử chỉ động được tổng hợp đầy đủ trong Bảng 4.



Hình 0.6: Ma trận nhầm lẫn của mô hình phân loại cử chỉ động

Lớp cử chỉ động	Precision	Recall	F1-Score	Support
Swipe Left	0.80	0.79	0.79	768
Swipe Right	0.88	0.80	0.84	758
Clockwise Circle	0.79	0.94	0.85	793
Counter Clockwise Circle	0.89	0.94	0.91	783
Swipe Down	0.85	0.88	0.86	796
Swipe Up	0.80	0.90	0.85	791
Triangle	0.87	0.74	0.80	806
Infinity	0.83	0.78	0.81	782
Z-Shape	0.80	0.89	0.84	785
S-Shape	0.90	0.71	0.79	785
Độ chính xác toàn cục (Accuracy)	0.84			7.847

Bảng 4: Báo cáo kết quả phân loại chi tiết (Classification Report) của mô hình cử chỉ động

0.4.2 Hiệu năng thời gian thực và Độ trễ hệ thống

Hệ thống được đánh giá hiệu năng thông qua hai chỉ số quan trọng là tốc độ xử lý khung hình (FPS) và tổng độ trễ phản hồi (latency). Về tốc độ xử lý (FPS), khi chạy luồng video gốc ở độ phân giải Full HD (1920×1080), MediaPipe gây quá tải cho CPU khiến tốc độ giảm xuống còn 18 FPS và xuất hiện hiện tượng giật lag rõ rệt; tuy nhiên, khi giảm độ phân giải xuống mức HD (960×540), tốc độ xử lý đã tăng lên và duy trì ổn định ở mức 30 - 32 FPS, đáp ứng yêu cầu cơ bản trong điều kiện thử nghiệm. Về độ trễ hệ thống, do Python và Unity chạy cục bộ trên cùng một máy tính, độ trễ truyền nhận qua socket UDP ở mức thấp trong quá trình thử nghiệm. Tổng độ trễ toàn trình, bao gồm thời gian camera thu ảnh, MediaPipe trích xuất khớp, Python suy luận cử chỉ và Unity cập nhật hiển thị, được ghi nhận dao động trong khoảng 35 – 45 ms. Mức trễ này chưa tạo ra hiện tượng gián đoạn rõ rệt khi quan sát và thao tác thử nghiệm.

0.4.3 Đánh giá hiệu quả làm mịn của bộ lọc One Euro Filter

Hiệu quả chống rung nhiễu của bộ lọc One Euro Filter được đánh giá thông qua quan sát độ ổn định của vị trí đầu ngón trỏ khi người dùng giữ tay tương đối cố định trước camera. Khi chưa áp dụng bộ lọc, tọa độ đầu ngón trỏ có thể dao động do nhiễu từ camera và sai số ước lượng landmark, khiến lười dao trong game FruitNinja bị rung nhẹ. Khi áp dụng One Euro Filter với các tham số $f_{c,min} = 1.0$ Hz và $\beta = 0.4$, dao động tọa độ được giảm đáng kể, giúp chuyển động của lười dao mượt hơn trong quá trình thử nghiệm. Tuy nhiên, hiệu quả của bộ lọc vẫn phụ thuộc vào điều kiện ánh sáng, chất lượng camera và tốc độ di chuyển tay của người dùng.

Kết chương

Chương 3 đã trình bày quá trình hiện thực hóa hệ thống từ việc đóng gói truyền dữ liệu qua UDP, thiết kế hệ xương tay ảo 3DHandModel và trò chơi tương tác FruitNinja, đến việc phân tích kết quả thực nghiệm cho thấy hệ thống đạt mức độ ổn định và độ chính xác phù hợp trong phạm vi thử nghiệm của đề tài. Tiếp theo, Chương 4 sẽ đưa ra những kết luận tổng kết và định hướng nghiên cứu phát triển tiếp theo của đề tài.